

The Reasoning Gap in Automated Vulnerability Detection: A Comprehensive Benchmark of LLMs, SAST, DAST, and Graph-based Approaches

1st Medhat Hassan Ali
Cairo, Egypt
Eng.MedhatHassanAli@gmail.com

2nd Youssef Sakr
Cairo, Egypt
Eng.Youssef.Sakr@gmail.com

3rd Abdullah Eissa
Cairo, Egypt
abdullaheissa588@gmail.com

4th Mohammed Aljazairy
Cairo, Egypt
mohammedaljazairy@icloud.com

5th Hisham Salah Rashad
Computer Engineering Department
Arab Academy for Science and Technology
Cairo, Egypt
hishamsalah@aast.edu

Abstract—Security vulnerabilities and exploit methodologies have evolved rapidly in recent years. Effective vulnerability identification and remediation in complex C/C++ ecosystems remain a critical challenge that outpaces traditional remediation strategies. This paper bridges the gap between traditional Application Security Testing (AST) and emerging AI-driven techniques. Specifically, we critically evaluate the efficacy of Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) alongside Deep Learning (DL) and Large Language Models (LLMs), focusing on their comparative ability in identifying security flaws. Our analysis scrutinizes benchmark integrity, quantifying significant overlap in real-world datasets, and dissects the “Reasoning Gap” in LLMs where open-weight models exhibit high recall but suffer from critical signal-to-noise issues. Our findings demonstrate that graph-based approaches consistently outperform text-based and traditional methods by capturing deeper semantic context. This paper delineates the limitations of current methodologies and establishes a framework for integrating AI-driven insights with structural analysis for next-generation vulnerability detection.

Index Terms—Vulnerability Detection, Static Analysis, AI for Security, Dynamic Analysis, Graph Neural Networks, Large Language Models, Code Property Graph, Automated Vulnerability Detection

I. INTRODUCTION

In recent years, security vulnerabilities and exploits have increased at an alarming rate. A source code vulnerability is a specific security flaw or what appears to be a defect in the code, which can be shown to be exploitable under certain circumstances. The impact of these vulnerabilities is severe and widespread, posing risks not just to code, but to the critical infrastructure and services people rely on daily. To put it simply, these flaws act as open doors for attackers to steal sensitive data, crash systems, or take control of devices. The 2025 Open-Source Security and Risk Analysis Report by Synopsys revealed that 81% of assessed commercial codebases containing open-source components exhibited high or critical-

risk vulnerabilities. As the landscape of exploit methodologies expands daily, effective vulnerability identification and remediation have become increasingly challenging.

Using the two approaches, AI-Driven Vulnerability Detection [1]–[3] and Application Security Testing (AST) techniques, this paper presents a thorough analysis of methods for finding vulnerabilities in C/C++ software programs. In particular, we examine Large Language Models (LLMs), graph-based models, and Deep Learning (DL) models in conjunction with conventional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) to assess how well they detect security vulnerabilities.

We also examine the Real and Artificial datasets frequently used in AST evaluation and focus on the vulnerability coverage capabilities of key structural representations, including Code Property Graph (CPG), Abstract Syntax Trees, Control Flow Graphs (CFG), and Program Dependence Graphs (PDG). Furthermore, we compare the advantages and disadvantages of query-based (Q-SAST) and non-query-based (NQ-SAST) tools.

The main contributions of this paper are summarized as follows:

- **Comprehensive Methodology Review:** We analyze the efficacy of AI-Driven, Graph-based, and AST approaches for C/C++ vulnerability detection.
- **LLM Benchmarking & The “Reasoning Gap”:** We evaluate state-of-the-art LLMs (including GPT-4o mini) across binary and multiclass tasks, uncovering a “Reasoning Gap” where open-weight models struggle with precision despite high recall.
- **Dataset Integrity Analysis:** We perform a critical comparison of prominent datasets (MegaVul and BigVul), quantifying data duplication to expose quality issues in current benchmarks.
- **Graph-based Superiority:** We demonstrate that graph-based methods consistently outperform sequence-based

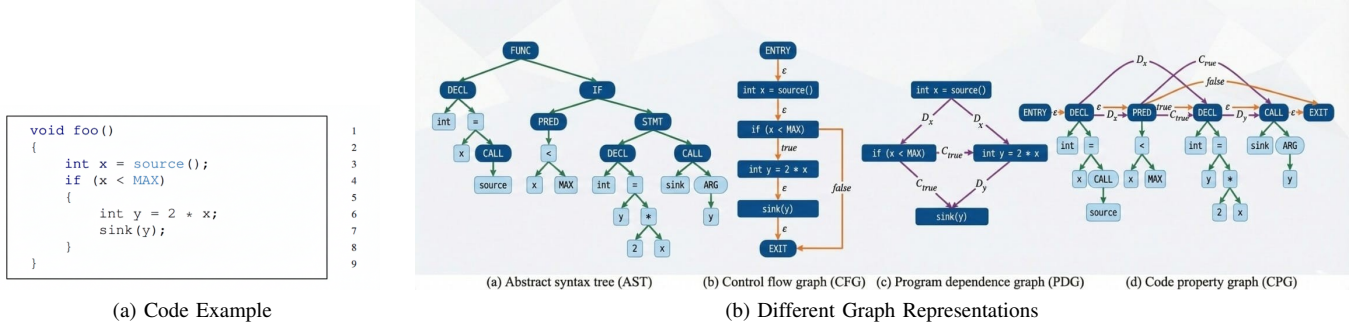


Fig. 1: Code Example in Different Graph Representation

approaches, providing deeper semantic insights for vulnerability detection.

- **Roadmap for Future Research:** We identify systematic limitations in current methodologies and outline specific development areas to drive continuous improvement in automated detection.

The remainder of this paper is organized as follows: Section II details the methodologies, including graph types and coverage, AI models, source code analysis tools, and datasets. Section III presents the experimental results across AI models, datasets, static analyzers, and dynamic analyzers. Section IV discusses the implications of these results, highlighting the limitations of existing methods and defining optimal deployment scenarios for each methodology. Finally, Section V concludes the study and outlines future research directions.

II. METHODOLOGY

A. Source Code Analysis Tools

1) *Static vs. Dynamic Analysis:* static approaches analyze variable ranges, control flow, and data dependencies in source code before running it, but they cannot detect vulnerabilities appearing only in specific runtime scenarios (e.g., clang) [4]. To monitor runtime behavior, dynamic analysis [4] runs programs with concrete inputs, but it often suffers from limited code coverage in complex systems (e.g., Valgrind [5]). Hybrid analysis combines static and dynamic approaches; all of the above primarily rely on known vulnerability patterns. There are evident advantages of using dynamic analysis for C/C++ security testing. Besides supporting binary instrumentation without compilation, and allowing for safer handling of pointer aliasing and locating memory leaks [6], it provides good evidence of exploitability through our Proof of Vulnerability (PoV), which is the proof an apparent defect is exploitable [7]–[9].

2) *Variant vs Classical Static Analyzers:* Classical static analyzers perform rule-based, deterministic checks to enforce correctness and coding standards. Variant static analyzers employ graph representations, probabilistic reasoning, and machine learning to identify complex vulnerabilities. These tools adapt to evolving code patterns and scale to large codebases.

TABLE I: Comparison of Static Analysis Tools

Category	Tool Name	User Queries	CWE Cov.
Q-SAST	CodeQL	Yes	146
	SonarQube	No	57
	Joern	Yes	≈ 146
	Semgrep	Yes	NA
NQ-SAST	Mend	No	148
	Clang	No	NA
	Coverity	No	101

3) *Query-based static tools (Q-SAST) and non-query-based static analysis tools (NQ-SAST):* Query-based SAST (Q-SAST) tools for example CodeQL convert code into queryable structures, enabling flexible and deep vulnerability analysis. In contrast, non-query-based SAST tools like Clang rely on predefined rules and are easier to deploy but less adaptable. As shown in Table I [10]–[16], Q-SAST tools are more effective at detecting semantically complex vulnerabilities [8].

B. Graph Types and Coverage

1) *Graph Types Used for Source Code Modeling:* Fig.1 shows Source code modeling relies on various graph representations to capture program semantics, including the Abstract Syntax Tree [17], CPG [12] and PDG [18]. These distinct models are often integrated into the Code Property Graph (CPG), which unifies syntax, control flow, and data dependencies to facilitate robust vulnerability detection [12].

Vulnerability Types	AST	AST + PDG	AST + CFG	CPG
Memory Disclosure	-	-	-	✓
Buffer Overflow	-	✓	-	✓
Resource Leaks	-	-	-	✓
Design Errors	-	-	-	✓
Null Pointer Dereference	-	-	-	✓
Missing Permission Checks	-	✓	-	✓
Race Conditions	-	-	-	✓
Integer Overflows	-	-	✓	✓
Division by Zero	-	-	✓	✓
Use After Free	-	-	✓	✓
Integer Type Issues	-	-	-	✓
Insecure Arguments	✓	-	-	✓

Legend: ✓ = Covered, - = Not Covered.

Fig. 2: Graph Coverage for Vulnerability Types

2) *Graph Coverage of Various Code Representations to Model Vulnerability Types*: Vulnerabilities can be categorized by required graph information:

- **Syntax-only vulnerabilities**: rely on ASTs but often cause false positives.
- **Control-flow vulnerabilities**: require CFGs and offer improved precision but limited data dependency modeling.
- **Taint-style vulnerabilities**: depend on combined syntax, control, and data-flow analysis within a CPG and provide the most expressive detection capability.

the flow of attacker-controlled data through the program and identify cases where proper sanitization is missing. According to Fig. 2 [12], [18], which outlines the capabilities of various graph types, the specific coverage for different vulnerability classes is detailed.

C. AI Models and Tools

1) *Devign*: As illustrated in Fig. 3, Devign [19] input is composite semantic graph (Devign main module is GNN models [20]). Its architecture consists of three stages: **Graph Embedding Layer** to encode semantics **Gated Graph Recurrent Layers** to aggregate neighbor information and learn node features and **Conv Layer** that extracts representations for graph-level binary classification (Vulnerable/Safe).

2) *Ample*: Operates in two phases shown in Fig. 4. First, **Graph Simplification** condenses the code structure graph using Type-based (TGS) and Variable-based (VGS) rules to merge redundant nodes. Second, **Enhanced Graph Representation Learning** employs an Edge-aware GCN with multi-head attention, followed by a Kernel-scaled convolution module to capture long-range dependencies for vulnerability detection [21].

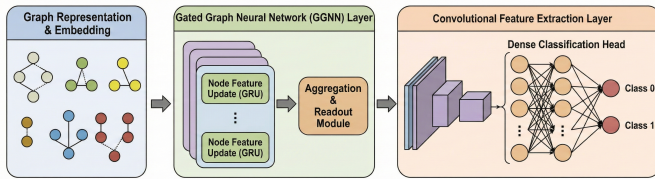


Fig. 3: Devign Architecture

```

System role:
You are cyber security engineer and want to read C
code and detect the vulnerability CWE number in the
code

User Prompt:
Look at the following C code and print the type of
the vulnerabilities and the type of CWE in this code
if and only if exists, only print the CWE number
without any explanation and without writing the code
again, if there is no vulnerability print secure

Input:
[C/C++ source code snippet]

```

Listing 1: Zero-shot Prompt used for LLM Evaluation

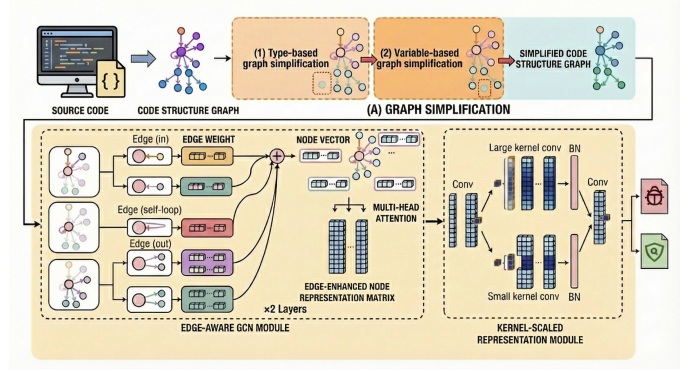


Fig. 4: Ample Architecture

3) *Large Language Models*: We evaluated several state-of-the-art LLMs, including Qwen2.5 0.5B parameters [22], Llama3.2 1B parameters [23], Gemma2 2B parameters [24], Deepseek v3.2 671 billion parameters [25], and GPT-4o mini [26]. The models were queried using a generation-based approach with default configurations (e.g., Temperature $T = 1$) and a specific system prompt, shown in Listing 1, to identify Common Weakness Enumeration (CWE) identifiers. Full implementation details and evaluation scripts are available in our GitHub repository [27]. Evaluation was conducted in two modes: **Binary**, where outputting any valid CWE-ID implies vulnerability and **Multiclass**, assessing the model's ability to predict the exact CWE-ID using micro-averaging.

D. Datasets

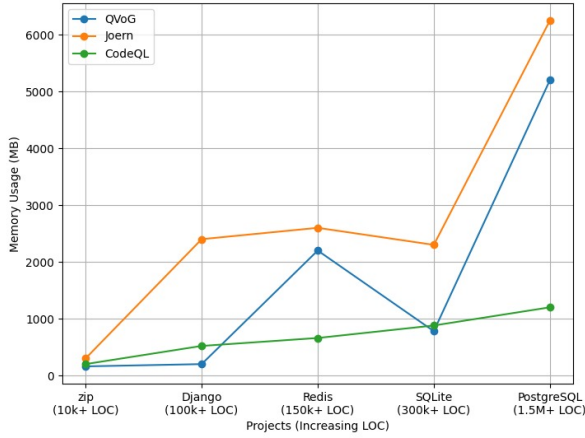
Artificial Dataset vs Real Dataset: Datasets used in software analysis can be categorized into two types: artificial datasets and real datasets. Artificial datasets consist of artificially generated code, created for specific purposes such as evaluating security scanners or testing security analysis techniques. In contrast, real datasets contain natural code which originates from actual software projects used in production environments. Examples Apache Web Server [28] or Microsoft Word. In general, the average Lines of Code (LOC) in real-world software projects tend to be significantly larger than those found in artificially generated.

1) Real Dataset:

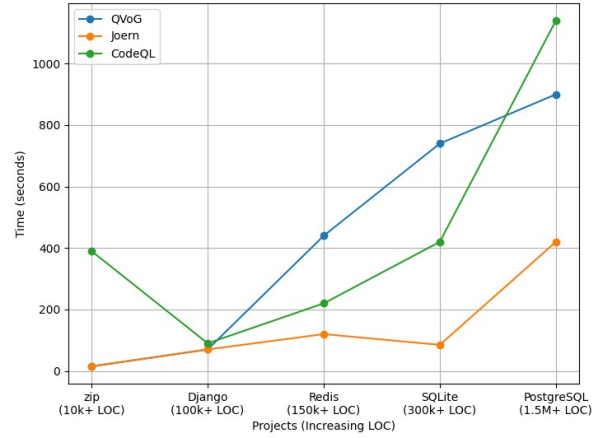
a) *MegaVul*: is a large, high-quality real-world dataset for vulnerability detection research. It contains 353,873 C/C++ functions collected across 1,062 repositories. The dataset covers 8,476 CVEs and 176 CWE-ID reported between January and 2024, enabling both text-based and graph-based detection models [29].

b) *BigVul*: is a real-world vulnerability dataset consisting of 188,636 C/C++ functions extracted from 4,058 vulnerability-fix commits. It includes 3,539 CVEs and 92 CWE-ID disclosed between 2002 and 2019 [30]. A comparison of the two datasets is shown in Table II [29], [30].

c) *Magma Dataset*: summarizes the Magma benchmark suite, which consists of seven widely-used open-source C/C++ targets such as libpng, openssl, and php [31]. The dataset



(a) CPG Extraction Memory Usage vs Project Size



(b) CPG Extraction Time vs Project Size

Fig. 5: Efficiency of CPG Extraction comparison. (a) Memory consumption for different tools. (b) Execution time for extraction.

TABLE II: Comparison of MegaVul and BigVul

Features	Mega-Vul	Big-Vul
Number of Repositories	1062	310
Number of CVE IDs	8476	3539
Number of CWE IDs	176	92
Number of Commits	9288	4058
Number of Vul/Non-Vul Function	17975/335898	10900/177736
Vul Ratio(%)	5.35	6.13
Success Rate of Graph Gen.	87%	None

includes 118 forward-ported vulnerabilities spanning multiple file formats and drivers. The targets are selected to incorporate challenging features such as magic values, recursive parsing, compression, and global states, which commonly hinder dynamic analysis. This design enables rigorous evaluation of fuzzers on realistic software behaviors.

d) Devign Dataset: is a manually labeled dataset derived from four large open-source projects. It contains 58,965 function graphs, including 27,652 vulnerable and 31,313 non-vulnerable functions collected from 23,355 vulnerability-fix and 25,332 non-fix commits [19]. Table III provides a project-level summary of the dataset [19].

e) Ample Dataset: is a composite dataset built from FFMpeg+Qemu, Reveal, and the Fan et al. dataset, totaling over 219,800 samples. The dataset exhibits vulnerability ratios

TABLE III: Devign Dataset

Project	Commits	VFCs	Non-VFCs	Graphs	Vul	Non-Vul
Linux Kernel	12811	8647	4164	16583	11198	5385
QEMU	11910	4932	6978	15645	6648	8997
Wireshark	10004	3814	6190	20021	6386	13635
FFmpeg	13962	5962	8000	6716	3420	3296
Total	48687	23355	25332	58965	27652	31313

respectively 45.02%, 9.16%, and 5.88% enabling evaluation under varying class imbalance conditions [21]. Key statistics are summarized in Fig. 6 [21].

2) Artificial Dataset:

a) Juliet: The **Juliet Test Suite v1.3** [32], developed by the NSA's Center for Assured Software, is a synthetic C/C++ dataset containing 64,099 labeled test cases covering 118 CWE-ID. It is widely used for evaluating SAST tools [33].

III. RESULTS

A. Static Analyzer Tools Results

1) Efficiency of CPG Extraction: To assess the efficiency and scalability of various static analysis tools, the researchers conducted tests across projects ranging from 10,000 to over

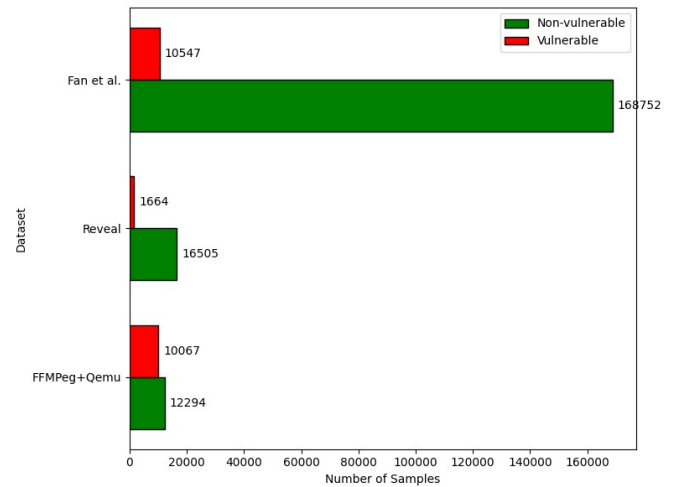


Fig. 6: Ample Dataset

TABLE IV: Joern Default Queries Evaluation on Balanced Benchmark Dataset

CWE ID	Cases	TP	FN	FP	TN	Prec	Rec	Acc
CWE-119	3440	1	1719	2	1718	33.33%	0.06%	49.97%
CWE-120	884	64	378	22	420	74.42%	14.48%	54.75%
CWE-134	60	2	28	0	30	100.00%	6.67%	53.33%
CWE-190	1526	39	724	14	749	73.58%	5.11%	51.64%
CWE-362	1110	4	551	0	555	100.00%	0.72%	50.36%
CWE-252	84	1	41	1	41	50.00%	2.38%	50.00%

1.5 million lines of code. Each tool was executed five times to ensure consistency and reliability of the measurements.

The results indicate that Joern demonstrates the best execution time across all scenarios. However, this superior time performance comes at the cost of high memory consumption, attributable to the extensive number of nodes and edges generated during the CPG creation. QVoG [34] exhibits a more efficient memory consumption compared to both Joern [12]

and CodeQL [10], primarily due to its reduced requirement for cross-file analysis. In contrast, CodeQL achieves better memory efficiency in specific cases but remains the slowest among the evaluated tools in CPG extraction. According to Fig. 5, which details extraction times and memory usage, these findings highlight the trade-offs between execution speed and memory utilization.

2) Real Dataset Results:

a) *Balanced Benchmark Dataset:* serves as a benchmark for evaluating vulnerability detection techniques using Joern’s default queries. This dataset contains 7,104 functions sampled from MegaVul, with an evenly split vulnerability distribution 50% of the functions are identified as vulnerable. It spans across 6 distinct CWE-IDs, providing a focused yet representative sample for default SAST performance analysis. Our evaluation of JoernV4 default queries on the Balanced Benchmark Dataset was conducted using the maximum number of vulnerabilities for each CWE-ID within the MegaVul dataset to ensure a solid evaluation. Each CWE result was filtered exclusively based on its corresponding queries according to our query-to-CWE mapping, as no official mapping has been provided by Joern’s authors. Our findings reveal that the number of True Positives (TPs) is exceedingly low across all CWE-IDs, accompanied by a relatively high False Positive (FP) rate and an overwhelming number of True Negatives (TNs). A detailed manual review of Joern’s default query Scala code uncovered significant limitations, including restricted coverage of CWE patterns, minimal to no custom function implementation analysis (a crucial factor in real-world software projects), and an absence of taint analysis in many queries necessary to mitigate false positives. A consistent pattern observed across all evaluated CWE-IDs is the alarmingly low number of TPs, which directly leads to exceptionally low recall rates (e.g., 0.06% for CWE-119 and a maximum of 14.48% for CWE-120). While precision

metrics exhibit variability, high precision in some cases (such as CWE-134 and CWE-362) is due to an insufficient number of test cases rather than effective detection. The

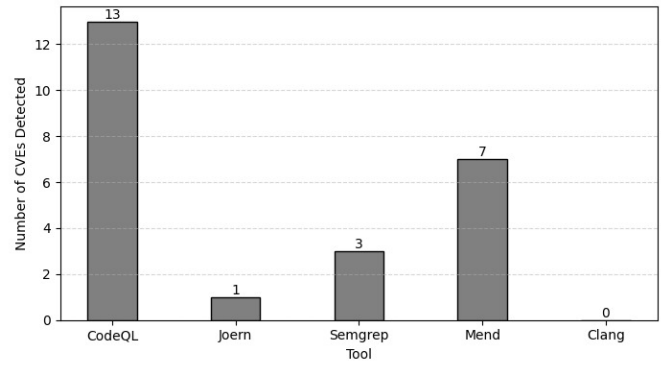


Fig. 7: Performance of Static Analysis Tools on Real-World CVEs

considerable number of TNs contributes to accuracy scores between 50-55% however, in the context of the datasets, such accuracy figures—coupled with the extremely low recall—suggest limited practical applicability. According to Table IV [12], which shows the precision, recall, and accuracy metrics for each CWE ID, these factors demonstrate that Joern’s default queries, in their current form, struggle significantly in detecting complex vulnerability patterns within real-world codebases.

b) *Standard Query Effectiveness:* Results CodeQL using its 46 default queries initially detected 13 CVEs. Joern detected only 1 CVE (Integer overflow in Jasper 1.0.25, CVE-2016-10261). Semgrep reported 3 CVEs, all related to the misuse of memcpy. Clang detected zero CVEs in the REAL dataset. Mend detected seven CVEs. However, the paper investigated them and found that Mend likely uses specific, brittle string-level signatures. For example, it detected CVE-2019-13135 in ImageMagick, but after the researchers renamed the variable “BImgBuff” to “k”, Mend failed to detect it. This suggested Mend’s approach was not based on deep semantic understanding for these CVEs. Manual Enhancement improved CodeQL’s total detection count on the real dataset rose from 13 to 30 CVEs (which is 130.77% improvement). This involved tailoring queries—such as making existing CVE-specific queries more restrictive by adding checks for sanitizing to reduce false positives or extending standard CWE queries to cover specific unhandled arithmetic operations (like integer left-shifts) to reduce false negatives. According to Fig. 7, which lists the number of CVEs detected by each tool, the performance results are presented above.

c) *Artificial Dataset Results:* **Juliet Dataset Results and Manual Review of Defects,** the team manually inspected 1,288 pull requests to classify and analyze each FP, FN, or analysis failure (AF) for CodeQL, Joern, and Semgrep. The research team didn’t just run the tools on test code. They went a step further to understand how these tools themselves have been fixed and improved over time by their developers and community. A key finding is that Q-SAST tools demonstrate superior vulnerability detection capabilities

TABLE V: Performance Evaluation of SAST Tools on the Juliet Dataset

CWE	#I	CodeQL			Joern			Semgrep			SonarQube			Mend			Coverity			Clang		
		TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN
78	1720	308	0	1412	196	132	1524	1374	73	273	8	231	1712	0	0	1720	204	114	1520	24	107	1696
121	4944	612	0	4332	863	1429	3152	3122	1815	7	1017	3529	0	0	0	4944	1707	530	3237	365	954	4579
131	2574	1230	113	1231	196	0	2378	396	85	2178	16	259	2385	0	0	2574	183	254	2136	38	146	2390
190	3960	3393	51	567	567	189	3204	1795	159	1790	139	1790	3031	0	0	3960	2040	898	1920	98	120	3742
191	2952	1866	0	1086	170	132	2676	1373	147	2676	60	139	2753	0	0	2952	1512	611	1140	140	85	2786
253	540	0	0	540	342	0	198	306	0	234	54	212	496	0	0	540	0	70	540	0	0	540
367	36	0	0	36	0	0	36	0	0	36	0	0	36	0	0	36	36	0	0	0	0	36
416	460	306	0	154	42	0	418	450	0	10	360	913	117	0	0	460	336	570	124	216	262	174
457	902	130	0	772	120	0	782	9	0	893	30	100	872	0	0	902	49	502	351	80	112	710
476	330	218	0	112	185	0	145	40	0	290	198	278	143	0	0	330	138	108	188	108	108	114
570	16	0	0	16	0	0	16	0	0	16	0	0	16	0	0	16	0	0	16	0	0	16
676	18	18	0	0	18	0	0	0	0	18	0	7	11	0	0	18	0	0	18	0	0	18
Total	18500	8439	2575	10061	5080	6180	13420	6599	14291	11882	2775	9186	15601	0	0	18500	6548	4196	11952	1258	1963	16861

TABLE VI: Mean Bug Survival Times (Reach vs. Trigger) Over a 24-Hour Period On Magma Dataset

Bug ID	moptafl		honggfuzz		afl++		aflfast		fairfuzz		symcc afl		Mean	
	R	T	R	T	R	T	R	T	R	T	R	T	R	T
AAH037	10.00s	20.00s	10.00s	10.00s	10.00s	45.50s	5.00s	15.00s	5.00s	15.00s	10.00s	25.50s	7.86s	20.86s
AAH041	15.00s	21.00s	10.00s	10.00s	15.00s	48.00s	10.00s	15.00s	10.00s	15.00s	15.00s	30.00s	12.14s	22.00s
AAH003	10.00s	16.00s	10.00s	11.00s	10.00s	15.00s	5.00s	10.00s	5.00s	10.00s	10.00s	1.58m	7.86s	23.86s
JCH207	10.00s	1.12m	5.00s	1.57m	10.00s	1.94m	5.00s	2.05m	5.00s	1.60m	10.00s	1.42m	7.14s	1.62m
AAH056	15.00s	14.57m	10.00s	14.43m	15.00s	19.49m	10.00s	13.07m	10.00s	11.27m	15.00s	8.17m	12.14s	14.11m
AAH015	32.50s	1.57m	10.00s	13.50s	27.00s	17.50m	1.18m	34.59m	52.00s	10.84m	15.07m	1.02h	2.76m	19.55m
AAH055	15.00s	40.86m	10.00s	2.71m	15.00s	3.62h	10.00s	25.01m	10.00s	2.24h	10.00s	6.36h	12.14s	2.26h
AAH020	5.00s	2.32h	5.00s	2.12h	5.00s	31.62m	5.00s	2.01h	5.00s	55.17m	5.00s	11.22h	5.00s	2.85h
AAH052	15.00s	3.17m	15.00s	14.10m	15.00s	45.03m	10.00s	3.94h	10.00s	10.56h	10.00s	12.02h	12.86s	3.95h
AAH032	15.00s	3.38m	5.00s	2.06m	15.00s	1.65h	10.00s	3.22h	10.00s	34.19m	10.00s	9.67h	11.43s	4.02h
MAE008	15.00s	1.42h	10.00s	9.73h	15.00s	1.44m	10.00s	1.14m	10.00s	1.54m	10.00s	12.08h	3.43h	6.76h
AAH014	12.48h	12.48h	24.00h	24.00h	13.06h	13.06h	6.34h	6.34h	24.00h	24.00h	18.46h	18.46h	15.57h	15.57h
JCH201/AAH007	15.00s	14.65h	10.00s	24.00h	5.00s	57.00s	15.00s	19.48h	15.00s	24.00h	10.00s	10.00s	12.14s	14.02h
AAH008	15.00s	16.51h	10.00s	3.65h	15.00s	23.40h	10.00s	19.44h	10.00s	19.66h	10.00s	15.28h	12.14s	17.34h

and its ongoing improvement and adaptation to new threats due to their query-driven nature, which allows users to define and refine vulnerability patterns. While initial, *Standard Query Effectiveness* performance is moderate, the effectiveness of Q-SAST tools significantly increases when human experts engage in a human-in-the-loop refinement process, adapting queries to enhance detection accuracy and decrease false positive rate. According to Table V [32], [33], which provides a detailed breakdown of tools performance across different CWEs, the results for the Juliet Dataset are shown.

B. Dynamic Analyzer Tools Results

Table VI presents the mean bug survival time over a 24-hour campaign, distinguishing between the time required to Reach (R) a vulnerable code path and the time required to actually Trigger (T) the fault. The data reveals a significant latency gap between these two states; for example, bug AAH015 was reached by moptafl in 32.50s but required 1.57m to trigger, while symcc afl reached it in 15.07m but took over an hour to trigger. The results highlight honggfuzz and moptafl as the most effective tools, frequently achieving the lowest time-to-trigger metrics. However, the prevalence of "24.00h" entries indicates that many bugs remained untriggered by standard mutational fuzzers like afl and afl++ within the time limit, proving that coverage does not guarantee detection.

C. AI models Results

1) *Devign*: Table VII [19] presents the performance results for Devign. The analysis demonstrates that the composite graph representation (Devign Comp) consistently outperforms single-edged graph models, achieving an accuracy improvement ranging from 0.11% to 3.75% and an average F1-score increase of 2.69%. Furthermore, the architectural inclusion of the Conv module allows Devign to significantly outperform the baseline Gated Graph Recurrent Network (GGRN). Overall, Devign achieves an average accuracy gain of 4.66% and an F1 gain of 6.37% compared to GGRN. This confirms that the composite graphs assist in learning better prediction models, and the Conv module effectively extracts relevant node features for superior graph-level classification.

2) *Ample*: Based on the analysis for the AI-based tools, AMPLE reports absolute returns of 4.75%, 6.86%, improvement in F1 score over the state-of-the-art on the three datasets, respectively, with the corresponding accuracy relative 7.63%, 16.48%, and 40.40%. According to Table VIII [21], [35], Ample results and comparisons with baseline methods are presented below.

3) LLM Models:

a) *Standard Distributed Dataset*: derived from the MegaVul dataset [29], serves as a benchmark for assessing the performance of vulnerability detection methods. This dataset comprises 35,387 functions, categorized based on their

TABLE VII: Devign Performance Analysis

Method	Linux Kernel		QEMU		Wireshark		FFmpeg		Combined		Max Diff		Avg Diff	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1
Devign (Abstract Syntax Trees)	80.24	84.57	71.31	65.19	79.04	64.37	66.89	73.83	69.21	69.99	3.95	7.88	2.33	3.37
Devign (CFG)	80.03	82.91	74.22	70.73	79.62	66.05	67.29	70.22	71.32	71.27	2.69	3.33	1.00	2.33
Devign (NCS)	79.58	81.41	72.32	68.98	79.75	65.88	65.29	68.89	70.82	68.45	2.29	4.81	1.46	3.84
Devign (DFG_C)	78.81	83.87	72.30	70.62	79.95	66.47	65.83	70.12	69.88	70.21	3.75	3.43	2.06	2.30
Devign (DFG_R)	78.25	80.33	73.77	70.60	80.66	66.17	66.46	72.12	71.49	70.92	3.12	4.64	1.29	2.53
Devign (DFG_W)	78.70	84.21	72.54	71.08	80.59	66.68	67.50	70.86	71.41	71.14	2.08	2.69	1.27	1.77
Devign (Comp)	79.58	84.97	74.33	73.07	81.32	67.96	69.58	73.55	72.26	73.26	-	-	-	-

TABLE VIII: Comparative Performance of AMPLE Against State-of-the-Art Baseline

Metrics	FFMPeg+Qemu				Reveal				Fan et al.			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
VulDeePecker	49.61	46.05	32.55	38.14	76.37	21.13	13.10	16.17	81.19	38.44	12.75	19.15
Russell et al.	57.60	54.76	40.72	46.71	68.51	16.21	52.68	24.79	86.85	14.86	26.97	19.17
SySeVR	47.85	46.06	58.81	51.66	74.33	40.07	24.94	30.74	90.10	30.91	14.08	19.34
Devign	56.89	52.50	64.67	57.95	87.49	31.55	36.65	33.91	92.78	30.61	15.96	20.98
Reveal	61.07	55.50	70.70	62.19	81.77	31.55	61.14	41.62	87.14	17.22	34.04	22.87
IVDetect	57.26	52.37	57.55	54.84	-	-	-	-	-	-	-	-
AMPLE	62.16	55.64	83.99	66.94	92.71	51.06	46.15	48.48	93.14	29.98	34.58	32.11

TABLE IX: Standard Distributed Dataset

Metric	Standard evaluation
Total number of functions	35,387
Number of Vul/Non-Vul Function	7015/28372
Vul Ratio (%)	19.82
Number of CWE IDs	142

vulnerability CWE-ID. Resulting in a vulnerability ratio of 19.82%. According to Table IX [29], which breaks down the function counts and vulnerability ratios, the details of the Standard Distributed Dataset are shown below.

b) Multiclass Evaluation: Table X [22]–[24], [26] presents the micro-averaged F1 scores for identifying specific CWE IDs. A sharp performance degradation is evident when moving from simple binary detection to specific classification. While Qwen and GPT-4o mini maintained robust F1 scores (0.5101 and 0.6180 respectively), Llama 3.2, Gemma 2, and Deepseek collapsed to F1 scores below 0.1. The Reasoning Gap chart illustrates this trend, highlighting how models with similar Binary performance diverge drastically when required to predict specific vulnerability type.

c) Binary Evaluation: Table XI details the classification performance between "Vulnerable" and "Safe" code. GPT-4o mini demonstrated the most balanced profile, achieving the highest Accuracy (69.06%) while maintaining a manageable False Positive Rate (FPR) of 23.6%. Conversely, the open-weight models (Deepseek v3.2 and Gemma 2) achieved superior Recall scores (greater than 88%) but at a prohibitive cost: they exhibited FPRs exceeding 90% and 87% respectively. As visualized in the Signal-to-Noise Ratio chart, this extreme

TABLE X: Multiclass Evaluation on Standard Distributed Dataset

Model	TP	FP	FN	Prec	Rec	F1
Qwen	19,372	21,195	16,015	47.75%	54.74%	0.5101
LLama 3.2	5,094	66,897	30,293	7.08%	14.40%	0.0949
Gemma 2	128	72,013	35,259	0.18%	0.36%	0.0024
Deepseekv3.2	3,483	32,054	31,904	9.80%	9.84%	0.0982
GPT4o mini	21,872	13,524	13,515	61.79%	61.81%	0.6180

TABLE XI: Binary Evaluation on Standard Distributed Dataset

Model	TP	TN	FP	FN	Prec	Rec	F1	Acc
Qwen	2230	19362	9010	4785	19.84%	31.79%	0.2443	61.02%
LLama 3.2	5876	4787	23585	1139	19.95%	83.76%	0.3222	30.13%
Gemma 2	6186	3509	24863	829	19.92%	88.18%	0.3250	27.40%
Deepseekv3.2	6784	2788	25584	231	20.96%	96.71%	0.3445	27.05%
GPT4o mini	2780	21660	6712	4235	29.29%	39.63%	0.3368	69.06%

over-prediction results in a scenario where actual vulnerabilities are buried under tens of thousands of false alarms.

4) Precision vs. Granularity in LLMS Evaluation:

a) Reasoning Gap: Fig. 8 shows a comparison of F1 stability. While all models show comparable capacity for binary detection, the open-weight models (excluding Qwen) fail to translate this into precise multiclass classification.

b) Signal-to-Noise: Fig. 9 disproportionate volume of False Positives (Red) compared to TP (Green) in models like Deepseek and Gemma highlights a severe risk of alert fatigue, rendering these models dangerous to deploy in automated security pipelines.

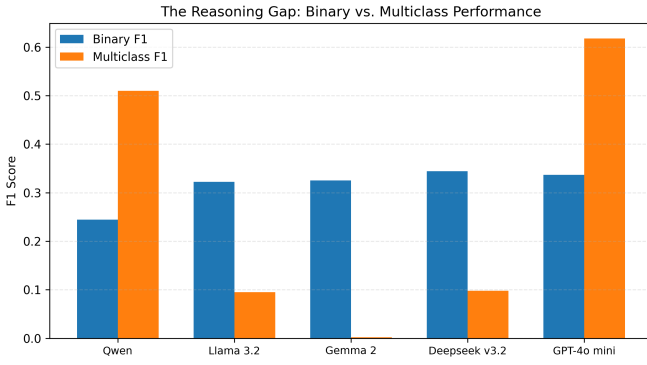


Fig. 8: The Reasoning Gap

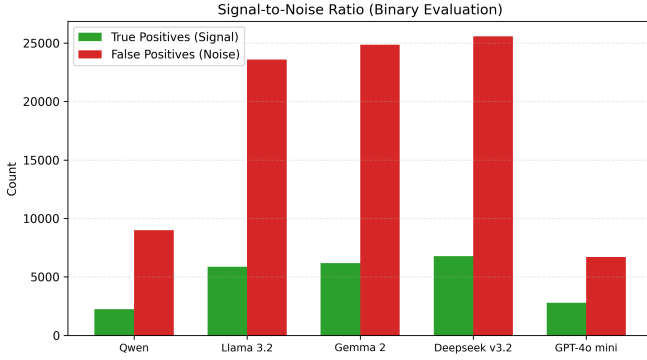


Fig. 9: Signal-to-Noise

IV. DISCUSSION

A. Key Findings and Approach Effectiveness

Based on our extensive evaluation, clear leaders and architectural advantages have emerged within each vulnerability detection methodology:

- **Static Analysis (SAST): CodeQL (Q-SAST)** demonstrated superior performance over non-query tools. Its effectiveness is significantly amplified through **manual query enhancement**, where security experts refine detection logic to target specific CVEs. This adaptability allows Q-SAST to identify semantically complex vulnerabilities in large code bases that rigid, traditional pattern-matching engines frequently miss.
- **Dynamic Analysis (DAST):** In fuzzing benchmarks, **honggfuzz** and **MOpt-AFL** consistently outperformed competitors. Their effectiveness stems from advanced mutation strategies that navigate complex execution paths, achieving the lowest time-to-trigger metrics for deep-state bugs that static analysis cannot reach.
- **AI-Driven Detection: Graph-based models** proved to be the most robust architecture, leveraging structures like the CPG to capture control and data dependencies that linear models miss. Among LLMs, **GPT-4o mini** yielded the excellent balance of Accuracy and F1-score.

- **Overall Architecture:** The Graph-based approach allows for **continuous improvement**. Future advancements lie in refining graph generation techniques to capture deeper semantic dependencies via scalable graph optimization and iteratively improving the detection methodologies (whether AI or AST) built upon these structural foundations.

B. Dataset Quality and Evaluation Metrics

Our analysis highlights that the integrity of the detection methodology is inextricably linked to **dataset quality**. The significant overlap found between benchmarks like MegaVul and BigVul underscores the need for rigorous deduplication to prevent data leakage. Furthermore, in real-world scenarios, the ratio of vulnerable to safe functions is highly imbalanced (with safe code predominating). Consequently, the **F1-score** emerges as the critical performance metric unlike Accuracy, which can be misleading in imbalanced datasets, the F1-score provides a true measure of a tool's ability to discriminate between vulnerable and safe code without being overwhelmed by false positives and false Negatives.

C. Limitations

1) **SAST:** The **static nature** of SAST provides comprehensive syntax, control, data flow and taint analysis but fails to interpret code and limited coverage of runtime behaviors issues like race conditions. **Undecidability** remains a core challenge; unlike dynamic analysis, SAST identifies potential vulnerabilities without generating a PoV, focusing instead on generic mining vulnerabilities in large-scale projects [36]. Furthermore, SAST struggles with **inter-procedural analysis**, where traversing function call graphs is computationally difficult, and lacks **external communication analysis**, failing to model interactions with system calls, APIs, or databases, which can obscure protocol or validation flaws [37].

2) **DAST:** The **dynamic nature** of DAST captures runtime behaviors like memory leaks and race conditions missed by static analysis and generate a Pov [6], [7], but introduces significant **performance overhead**. Tools like Valgrind utilize detailed instrumentation that is **resource intensive**, consuming substantial CPU and memory which impacts scalability [6], [32]. Furthermore, **time complexity** is dependent on input size and instrumentation extent, often leading to exponential execution slowdowns on large applications [6], [7].

3) AI Models:

a) **Text-Based Models:** Text-based architectures, particularly those built on the Transformer architecture, are constrained by a **fixed context window** (maximum input sequence length), requiring **truncation strategies** that lead to the **information loss** of critical logic for vulnerability detection. Treating code as a **linear sequence** is suboptimal for capturing **structural dependencies** and complex control flows compared to graph representations. Additionally, **domain adaptation** presents a hurdle, as standard natural language **tokenizers** often fail to parse programming syntax effectively without specialized pre-training on large-scale code.

b) *Graph-Based Models*: Graph-based architectures face **scalability challenges**. As codebases grow, their corresponding graph representations expand exponentially, often forcing developers to use **simplification techniques** that risk discarding important semantic details [21]. Additionally, these models introduce substantial **time complexity** due to the heavy preprocessing required to convert raw code into specific structures, such as Abstract Syntax Trees or CPG. This creates a **rigid dependency** on the graph type and the underlying parser [34].

4) *Datasets*: A significant limitation in current research is the **lack of a comprehensive, CWE-categorized real-datasets**. Furthermore, our MegaVul and BigVul **dataset overlap analysis** reveals **substantial duplication** between existing sources. MegaVul contains 353,873 entries (9,288 unique hashes) while BigVul holds 188,636 (4,058 unique hashes). Despite this, deduplication shows meaningful intersections: 4,190 vulnerable function rows share commit hashes. After duplicate removal, the unique hash intersection stands at 2,513, with 2,459 intersected CVE IDs. This indicates that 62% of BigVul’s unique hashes exist in MegaVul, whereas only 27% of MegaVul’s exist in BigVul. Crucially, 95% of these intersected hashes correspond to vulnerable functions, highlighting **strong vulnerability overlap** and emphasizing the need to refine dataset utility.

D. Usage Recommendations

Selecting the appropriate methodology requires balancing the TP rate against the False Positive Rate (FPR), while also considering the operational computational overhead. We propose the following deployment strategy based on performance capability and deployment feasibility:

1) *Complexity and Deployment Feasibility*: The practical adoption of automated vulnerability detection tools depends on computational overhead, integration capabilities, and reporting standards.

- **Computational Overhead**: Graph-based AI models exhibit high complexity per LOC due to the heavy preprocessing required to generate CPGs. Similarly, DAST incurs significant runtime overhead (up to 22x with Valgrind) and requires extended CPU time for fuzzing campaigns [5]. In contrast, SAST tools scale almost linearly with codebase size, while LLMs offer the lowest per-LOC overhead during inference, though training or fine-tuning remains resource-intensive.
- **SDLC & CI/CD Integration**: LLMs offer a “plug-and-play” deployment with zero pre-configuration, making them easy to adopt but difficult to tightly integrate into the SDLC as rigorous quality gates due to their non-deterministic output. In contrast, SAST tools like CodeQL integrate deeply into CI/CD workflows, enforcing security policies and breaking builds, but require significant pre-configuration, such as defining build environments and project-specific configuration files within the repository.

- **API Integration & Reporting**: LLMs excel in flexibility, offering JSON-formatted outputs and natural language explanations via standard APIs, facilitating easy integration into dashboards. Traditional SAST/DAST tools produce standardized formats (e.g., SARIF) but often require proprietary viewers or parsers to interpret complex data-flow paths effectively.

2) Performance:

- **Use AI-Driven Models (LLMs)**: For rapid auditing or assisting human developers. Models like Deepseek offer high TP rates (high recall), they suffer from massive FPR (noise). Conversely, GPT-4o mini offers a more balanced, conservative approach suitable for pre-filtering code.
- **Use SAST (CodeQL)**: For **large, evolving codebases** where the environment changes frequently. Its query-based nature allows teams to develop custom queries for specific CVE coverage, ensuring the tool adapts to new vulnerability patterns without requiring a tool upgrade.
- **Use DAST (Fuzzing)**: For applications that rely heavily on **external user input** and complex **memory allocations**. It is the only methodology capable of verifying runtime behavior and generating a PoV for issues like memory leaks and buffer overflows.

V. CONCLUSION AND FUTURE WORK

This work has offered a thorough assessment of the present state of the art in code vulnerability detection, with a specific focus on C/C++ systems, pinpointing a need for a paradigm shift away from linear approaches. Our analysis reveals that, although conventional static analyzers and modern large language models possess remarkable scalability, they often lack the semantic insight to segregate complex vulnerabilities from benign behaviors, resulting in high false positive rates. In contrast, graph-based methods, particularly those utilizing the CPG, demonstrate the greatest potential for representing complex control and data dependencies in practical scenarios. Therefore, the future of automated security verification must migrate toward a hybrid approach that merges the structural accuracy of Graph Neural Networks (GNNs) with the flexibility of Query-based static analysis.

Future research should focus on **enhancing AI efficiency** by adopting techniques like QLoRA [38] (Quantized Low-Rank Adaptation) from the PEFT [39] library to reduce the computational resources and carbon emissions associated with fine-tuning pre-trained models. Additionally, significant improvements can be achieved through **manual enhancement of Q-SAST tools**, where cybersecurity experts refine query logic to expand CWE coverage and lower False Positive Rates (FPR). Finally, the field must prioritize **integrating AI-driven insights with structural analysis**, creating next-generation frameworks that combine the reasoning capabilities of LLMs with the precision of graph-based representations to address the evolving complexity of modern exploits.

REFERENCES

- [1] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?” *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, 2022.
- [2] C. Liang, Q. Wei, J. Du, Y. Wang, and Z. Jiang, “Survey of source code vulnerability analysis based on deep learning,” *Comput. & Secur.*, vol. 148, p. 104098, 2025.
- [3] Z. Li, D. Zou, J. Tang, Z. Zhang, M. Sun, and H. Jin, “A comparative study of deep learning-based vulnerability detection system,” *IEEE Access*, vol. 7, pp. 103184–103197, 2019.
- [4] F. Tudela, J. R. Higuera, J. Bermejo, J. A. Montalvo, and M. Argyros, “On combining static, dynamic and interactive analysis security testing tools to improve OWASP top ten security vulnerability detection in web applications,” *Applied Sciences*, vol. 10, no. 24, p. 9119, 2020.
- [5] N. Nethercote and J. Seward, “Valgrind: A framework for heavyweight dynamic binary instrumentation,” in *Proc. ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2007, pp. 89–100.
- [6] K. Vorobyov, P. Krishnan, and P. Stocks, “A dynamic approach to locating memory leaks,” in *Proc. IEEE Int. Conf. Softw. Testing, Verif. Valid. Workshops*, 2013, doi: 10.1007/978-3-642-41707-8_17.
- [7] S. Ul Haq, Y. Singh, A. Sharma, et al., “A survey on IoT & embedded device firmware security: Architecture, extraction techniques, and vulnerability analysis frameworks,” *Discov. Internet Things*, vol. 3, no. 17, 2023, doi: 10.1007/s43926-023-00045-2.
- [8] Z. Li, Z. Liu, W. K. Wong, P. Ma, and S. Wang, “Evaluating C/C++ vulnerability detectability of query-based static application security testing tools,” *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4600–4618, 2024.
- [9] L. Zhang, Q. Zou, A. Singhal, X. Sun, and P. Liu, “Evaluating large language models for real world vulnerability repair in C/C++ code,” in *Proc. 10th ACM Int. Workshop Secur. Privacy Analytics (IWSPA)*, Porto, Portugal, 2024.
- [10] M. Shen, A. A. Pillai, B. A. Yuan, J. C. Davis, and A. Machiry, “Finding 709 defects in 258 projects: An experience report on applying CodeQL to open-source embedded software,” *arXiv preprint arXiv:2310.00205v2*, 2025.
- [11] J. García-Muñoz, M. García-Valls, and J. Escribano-Barreno, “Improved metrics handling in SonarQube for software quality monitoring,” in *Software Quality as a Strategic Driver*, Cham: Springer, 2016, pp. 637–646, doi: 10.1007/978-3-319-40162-1_50.
- [12] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *Proc. IEEE Symp. Secur. Privacy (S&P)*, Berkeley, CA, USA, 2014, pp. 590–604, doi: 10.1109/SP.2014.44.
- [13] L. Golovyrin, “Analysis of tools for static security testing of applications,” 2023, doi: 10.13140/RG.2.2.28580.45446/1.
- [14] B. Babati, G. Horváth, V. Májer, and N. Pataki, “Static analysis toolset with Clang,” in *Proc. 10th Int. Conf. Appl. Informat. (ICAI)*, 2018, pp. 23–29, doi: 10.14794/ICAI.10.2017.23.
- [15] V. Lenarduzzi, S. Lujan, N. Saarimaki, and F. Palomba, “A critical comparison on six static analysis tools: Detection, agreement, and precision,” *arXiv preprint arXiv:2101.08832*, 2021.
- [16] O. d. Moor et al., “Keynote address: .QL for source code analysis,” in *Proc. 7th IEEE Int. Working Conf. Source Code Anal. Manipul. (SCAM)*, Paris, France, 2007, pp. 3–16, doi: 10.1109/SCAM.2007.31.
- [17] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
- [18] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 3, pp. 319–349, 1987.
- [19] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, 2019, pp. 10197–10207.
- [20] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, 2009.
- [21] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J. M. Zhang, and Q. Liao, “Vulnerability detection with graph simplification and enhanced graph representation learning,” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Australia, 2023, pp. 2275–2286, doi: 10.1109/ICSE48619.2023.00191.
- [22] Qwen Team, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [23] A. Grattafioriet et al., “Llama 3: The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [24] Gemma Team, “Gemma: Open models,” *arXiv preprint arXiv:2403.08295*, 2024.
- [25] DeepSeek-AI et al., “DeepSeek-V3.2: Pushing the frontier of open large language models,” *arXiv preprint arXiv:2512.02556*, 2025.
- [26] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [27] Our Github, “Cpp-Vuln-Benchmark,” GitHub repository, 2026. [Online]. Available: <https://github.com/Cpp-Vuln-Benchmark>
- [28] D. Wibowo, H. Zidny, and F. Insani, “Apache web server security with security hardening,” *J. Soft Comput. Explor.*, vol. 4, no. 4, pp. 213–221, 2023, doi: 10.52465/joscex.v4i4.230.
- [29] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, “MegaVul: A C/C++ vulnerability dataset with comprehensive code representation,” *arXiv preprint arXiv:2406.12415*, 2024.
- [30] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proc. 17th Int. Conf. Mining Softw. Repos. (MSR)*, Seoul, South Korea, 2020, pp. 508–512, doi: 10.1145/3379597.3387501.
- [31] A. Hazimeh, A. Herrera, and M. Payer, “Magma: A ground-truth fuzzing benchmark,” in *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 4, no. 3, 2020, pp. 1–29.
- [32] T. Bolland and P. Black, “Juliet 1.1 C/C++ and Java test suite,” *Computer*, vol. 45, no. 10, pp. 88–90, Oct. 2012, doi: 10.1109/MC.2012.345.
- [33] A. Wagner and J. Sametinger, “Using the Juliet test suite to compare static security scanners,” in *Proc. 11th Int. Conf. Secur. Cryptography (SECURITY)*, Vienna, Austria, 2014, pp. 1–9.
- [34] Z. Liu, X. Zhong, X. Deng, S. Hong, X. Gao, and H. Sun, “Scalable defect detection via traversal on code graph,” *arXiv preprint arXiv:2406.08098*, 2024.
- [35] Z. Li, D. Zou, S. Xu, X. Ou, and H. Jin, “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018.
- [36] G. Acerbi, “Mining for vulnerabilities in embedded TCP/IP stacks with a set of static analysis queries,” M.S. thesis, Ca’ Foscari Univ. Venice, Venice, Italy, 2020.
- [37] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [38] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
- [39] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, “Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment,” *arXiv preprint arXiv:2312.12148*, 2023.